

Constructive Orthogonal Corridor Routing in Binary Space Partitions

Preprint v0.5

frogscooper

stemboy@posteo.com

<https://frogscooper.dev>

September 2026

Abstract

Binary space partitioning (BSP) recursively decomposes a region into cells and has long been used for spatial organization in graphics and geometric computation [1, 2]. In procedural generation, BSPs also provide a natural scaffold for placing rooms or other occupied regions inside leaf cells [6]. This preprint studies a narrower connection problem: constructing an orthogonal corridor between selected rooms by walking the same BSP partition that generated them, rather than invoking a separate global path search. The routing procedure advances one coordinate at a time, crosses adjacent BSP leaves in the direction of the target, and is designed to preserve a boundary coordinate while traversing non-endpoint leaves. We separate correctness into three claims: monotone termination, preservation of the intermediate-leaf boundary invariant, and finite-width collision avoidance. Once the boundary invariant is established, collision avoidance follows from a simple clearance argument: if every non-endpoint room is separated from its leaf boundary by at least δ , then a corridor of half-width $h < \delta$ cannot intersect that room. A Rust implementation provides a concrete realization of the algorithm. As a secondary demonstration rather than a primary contribution, the core geometry has also been generalized with const generics so the same code can be instantiated in arbitrary finite dimension. This version is intended for early public review. The complete implementation-level case proof of the boundary invariant remains an open item before journal submission. The literature review in this version compares the construction against prior BSP motion planning, rectilinear obstacle routing, guillotine partitions, and BSP-based dungeon corridor generation; it is not claimed to be exhaustive.

Keywords. binary space partitioning, orthogonal routing, rectilinear routing, procedural generation, computational geometry, collision avoidance

1 Introduction

Binary space partitioning was introduced as a hierarchical way to decompose space using recursively chosen separating hyperplanes [1]. BSP representations have subsequently appeared

in geometric computation and motion-planning settings [2]. Separately, procedural-content-generation literature describes spatial subdivision techniques, including BSP-style decompositions, as useful tools for constructing game spaces and other generated layouts [6].

The implementation that motivated this paper is a procedural dungeon generator. Rooms are placed strictly inside selected BSP leaves, and the leaves themselves provide a hierarchy of adjacent free-space regions. Room placement is therefore straightforward: containment follows directly from the leaf bounds. Connecting two selected rooms is less immediate. One option is to formulate a search problem over a graph or grid and use a general shortest-path method such as Dijkstra’s algorithm or A* [4, 5]. That approach is general, but it treats routing as a problem separate from the partition that already generated the geometry.

This paper studies a more specific question:

Can the existing BSP partition itself be used to construct an orthogonal corridor between selected rooms, while giving a direct geometric condition under which the corridor avoids every non-endpoint room?

The proposed algorithm is constructive rather than globally search-based. It walks through adjacent BSP leaves, changes one coordinate at a time, and uses the boundaries of intermediate leaves as the geometric structure along which the route is formed. The paper’s primary subject is this routing algorithm and its correctness conditions. The arbitrary-dimensional implementation is included only as a compact demonstration that the construction is coordinate-based rather than tied to the names x , y , and z .

The intended contribution is narrow. BSP motion planning and rectilinear path planning are established subjects [2, 3]; this manuscript does not claim otherwise. Instead, it isolates a particular route construction tied directly to the BSP that generated the occupied regions, together with an explicit finite-width clearance condition. The current claims are:

- (i) routing of a fixed coordinate terminates because progress is monotone across a finite partition;
- (ii) subject to the stated boundary-preservation invariant, every intermediate route segment stays on a boundary of the leaf it traverses; and
- (iii) if the corridor half-width is smaller than the minimum room-to-boundary clearance, the finite-width corridor cannot intersect a non-endpoint room.

The second claim is the central proof obligation. This preprint states the invariant explicitly and gives the supporting geometric argument, but the full implementation-level case split is deliberately marked incomplete rather than presented as finished.

2 Geometric Model

Definition 1 (Root box). *Let*

$$B = \prod_{i=1}^d [\ell_i, u_i] = \{x \in \mathbb{R}^d : \ell_i \leq x_i \leq u_i \text{ for every } i\},$$

where $\ell_i < u_i$ for every coordinate i . Here $\prod$ denotes the Cartesian product of the coordinate intervals; for example, in three dimensions it means $[\ell_1, u_1] \times [\ell_2, u_2] \times [\ell_3, u_3]$.

A BSP node stores an axis-aligned box. An internal node chooses an axis $a \in \{1, \dots, d\}$ and a split coordinate s strictly inside its interval on that axis. The two children replace $[\ell_a, u_a]$ with $[\ell_a, s]$ and $[s, u_a]$, respectively, while every other coordinate interval is unchanged. Repeating this operation produces a finite set of axis-aligned leaf boxes whose interiors are disjoint and whose union is B . This is the axis-aligned special case of the general BSP idea [1].

Definition 2 (Room). *For each occupied leaf $L = \prod_i [\ell_{L,i}, u_{L,i}]$, a room is an axis-aligned box*

$$R_L = \prod_{i=1}^d [r_{L,i}^-, r_{L,i}^+]$$

with

$$\ell_{L,i} < r_{L,i}^- \leq r_{L,i}^+ < u_{L,i}$$

for every i .

The strict inequalities give each room positive separation from every face of its leaf.

Definition 3 (Leaf clearance). *Define the L_∞ clearance of R_L from the boundary of L as*

$$\delta_L = \min_i \{r_{L,i}^- - \ell_{L,i}, u_{L,i} - r_{L,i}^+\}.$$

For a finite set of occupied leaves $\mathcal{L}$, let

$$\delta = \min_{L \in \mathcal{L}} \delta_L.$$

When all rooms are strictly inset, $\delta > 0$.

Definition 4 (Orthogonal route). *An orthogonal route is a polyline $\gamma = (p_0, \dots, p_k)$ in which each consecutive pair differs in at most one coordinate. Equivalently, every segment is parallel to a coordinate axis. Such axis-aligned routes are a standard object in rectilinear path planning [3].*

Definition 5 (Finite-width corridor). *For half-width $h \geq 0$, define the corridor induced by a centerline γ as the L_∞ thickening*

$$C_h(\gamma) = \{x \in \mathbb{R}^d : \text{dist}_\infty(x, \gamma) \leq h\}.$$

The L_∞ metric is used because thickening an axis-aligned segment in this metric produces an axis-aligned box, matching the corridor geometry generated by the implementation.

3 Constructive Routing Algorithm

The input to routing is a source room, a target room, and the finite BSP leaf partition. Endpoint selection chooses a point on the source room and a point on the target room that face the BSP split that induced the connection. In the original three-dimensional implementation these points are face-centered.

The routing state consists of a current point p , target point q , current leaf L , and the output sequence of orthogonal segments. For each coordinate i , the procedure moves p_i toward q_i . If q_i lies within the current leaf's interval on axis i , the route moves directly to q_i on that axis. Otherwise it moves to the appropriate boundary of the current leaf, crosses into an adjacent leaf sharing that boundary, and continues.

Listing 1: Coordinate-wise BSP routing, schematic pseudocode.

```
ROUTE(p, q, leaves):
  L ← leaf containing p
  for axis i in -1, ..., d:
    while p[i] != q[i]:
      if q[i] lies within L on axis i:
        append segment p → p with coordinate i set to q[i]
        p[i] ← q[i]
      else if q[i] < p[i]:
        b ← upper boundary of L on axis i
        append segment p → p with coordinate i set to b
        p[i] ← b
        L ← adjacent leaf across that boundary
      else:
        b ← lower boundary of L on axis i
        append segment p → p with coordinate i set to b
        p[i] ← b
        L ← adjacent leaf across that boundary
  return route
```

The production implementation includes an initial snapping phase and resolves an adjacent leaf using the crossing point's non-moving coordinates. Those details are important for the exact boundary-invariant proof but do not alter the monotonicity argument below.

4 Termination

Lemma 1 (Monotone coordinate progress). *Fix a coordinate i . During the portion of the construction that routes coordinate i , the value of p_i changes monotonically toward q_i and never moves away from q_i .*

Proof. Suppose first that $p_i < q_i$. The construction either sets p_i directly to q_i when the target coordinate lies inside the current leaf interval, or sets p_i to the upper boundary of the current leaf on axis i . In the latter case the route proceeds across that boundary to an adjacent leaf in the direction of q_i . No rule decreases p_i while this coordinate is being routed. The case $p_i > q_i$ is symmetric, using the lower boundary and never increasing p_i . $\square$

Lemma 2 (Finite traversal of one coordinate). *Routing a fixed coordinate i terminates after finitely many leaf transitions.*

Proof. The BSP contains finitely many leaves. During routing of coordinate i , the route crosses leaf boundaries only in the direction of q_i by the preceding lemma. It therefore cannot return to a previously crossed leaf by reversing direction along the same coordinate. Because only finitely many leaves can be encountered, only finitely many such transitions are possible. Eventually the current leaf's interval on axis i contains q_i , at which point the construction sets $p_i = q_i$. $\square$

Theorem 1 (Termination). *For finite d and a finite BSP partition, the coordinate-wise routing construction terminates after finitely many steps, provided that each required boundary crossing has a valid adjacent leaf.*

Proof. There are finitely many coordinates. By the previous lemma, routing each coordinate terminates after finitely many leaf transitions. A finite sequence of finite traversals is finite. $\square$

5 Boundary Invariant and Collision Avoidance

The collision argument separates centerline safety from corridor thickness. This distinction is useful because the geometric proof can be stated independently of the implementation's chosen corridor width.

Lemma 3 (Boundary-preservation invariant — proof obligation). *Let L be a non-endpoint leaf traversed by the route. During traversal through L , at least one coordinate of the route is fixed to a boundary value of L ; consequently the centerline segment inside L lies on ∂L .*

Proof sketch. A boundary crossing from one leaf to an adjacent leaf occurs at a shared axis-aligned face. At the crossing point, the coordinate normal to that face equals the corresponding boundary value of the new leaf. A subsequent orthogonal segment changes only one coordinate. Whenever the changed coordinate is not the face-normal coordinate, that boundary coordinate remains fixed, so the segment stays on the face. The implementation's initial snapping phase and coordinate-update order are intended to preserve such a pinned boundary coordinate as routing proceeds. A complete proof must check the exact state transitions for increasing and decreasing motion and for the cases in which the next target coordinate is either inside the current leaf interval or beyond the next boundary. That implementation-level case proof remains to be completed for the next version. $\square$

Lemma 4 (Clearance from an inset room). *Let R_L be a room in leaf L with clearance at least δ_L . If a route segment γ_L lies on ∂L , then*

$$\text{dist}_\infty(\gamma_L, R_L) \geq \delta_L.$$

Proof. Take any point $x \in \partial L$. Because x lies on the boundary of the axis-aligned box L , at least one coordinate j satisfies either $x_j = \ell_{L,j}$ or $x_j = u_{L,j}$. Every point $y \in R_L$ is inset from that same face by at least δ_L , so $|x_j - y_j| \geq \delta_L$. Therefore $\|x - y\|_\infty \geq \delta_L$. This holds for every $x \in \partial L$ and every $y \in R_L$, hence also for the subset $\gamma_L \subseteq \partial L$. $\square$

Theorem 2 (Finite-width non-intersection). *Assume the boundary-preservation invariant holds in every non-endpoint occupied leaf and let*

$$\delta = \min_L \delta_L > 0.$$

If the corridor half-width satisfies

$$0 \leq h < \delta,$$

then the finite-width corridor does not intersect any non-endpoint room.

Proof. Assume for contradiction that a non-endpoint room R_L intersects the corridor $C_h(\gamma)$. Then there exists a point $z \in R_L \cap C_h(\gamma)$. Since $z \in C_h(\gamma)$, by definition there exists a point $x \in \gamma$ such that

$$\|x - z\|_\infty \leq h.$$

Hence

$$\text{dist}_\infty(\gamma, R_L) \leq h.$$

But the boundary invariant and the preceding lemma imply

$$\text{dist}_\infty(\gamma, R_L) \geq \delta_L \geq \delta.$$

Combining the inequalities gives

$$\delta \leq h < \delta,$$

a contradiction. Therefore $C_h(\gamma) \cap R_L = \emptyset$ for every non-endpoint room R_L . $\square$

Corollary 1 (Global sufficient width bound). *If every generated room is inset from every face of its leaf by at least a known quantity $\delta_{\min}$, then any corridor with total axis-aligned thickness $2h$ satisfying $h < \delta_{\min}$ is safe from non-endpoint rooms, subject to the boundary invariant.*

6 Complexity

Let k be the total number of leaf transitions made by a route and let d be the dimension. If adjacency at a boundary crossing can be resolved in $O(1)$ time after preprocessing, the routing work is $O(k + d)$. If a crossing is resolved by querying a BSP of height H , the work becomes $O(kH + d)$. For a balanced tree with n leaves, $H = O(\log n)$, giving $O(k \log n + d)$. These bounds concern route construction only and do not include BSP generation or room selection.

The current prototype uses a simpler leaf-map scan in places, so its concrete running time can be worse than the abstract data-structure bound. This distinction is intentional: the correctness argument does not depend on a particular acceleration structure.

7 Implementation and Demonstration

The reference implementation is written in Rust. It uses seeded random generation for reproducibility, stores the BSP as a binary tree of axis-aligned tiles, constructs rooms inside selected leaf tiles, derives connections from the BSP hierarchy, routes those connections orthogonally, and exports three-dimensional geometry for visual inspection.

7.1 Arbitrary-dimensional implementation as a secondary demonstration

The initial implementation represented points with three named coordinates. To test whether the routing logic depended on specifically three axes, the core geometry was generalized using Rust const generics [10]:

```
pub struct PointN; const N: usize; (pub [i64; N]);
```

The same dimension parameter is propagated through rooms, tiles, BSP nodes, routing edges, and corridor construction. Component-wise arithmetic and indexing are generic, BSP splitting uses coordinate indices rather than an $X/Y/Z$ enumeration, and routing iterates over $0..N$.

This generalization is not presented as the principal result of the paper. It is an implementation-level demonstration that the route construction can be expressed using indexed coordinates and axis-aligned intervals rather than special-case three-dimensional code. The OBJ exporter remains a three-dimensional visualization layer; for $d > 3$, correctness checks operate directly on the N -dimensional data structures rather than relying on visualization.

8 Current Regression Evidence

The test suite is engineering evidence that the implementation matches several intended properties; it is not a substitute for the proof above. All tests listed below pass in the current development version.

The most important next tests are property-oriented rather than dimensional. They should directly check the boundary-preservation invariant, containment of every segment inside the root region, non-intersection with every non-endpoint room, and behavior at $h < \delta$, $h = \delta$, and $h > \delta$ over many seeded BSP instances.

9 Relationship to Existing Work

9.1 Binary space partitions and motion planning

Binary space partitioning was introduced in computer graphics as a hierarchical spatial decomposition for visibility ordering [1]. The decomposition itself is therefore established technology, not a contribution of this paper. Tokuta later studied motion planning directly with a BSP representation [2]. That work is especially important to distinguish from the present manuscript: it establishes that BSP cells can support motion-planning computations, so the novelty question here cannot be “can BSPs be used for routing?” The narrower question is whether the particular corridor construction used here—coordinate-wise traversal of the already-generated leaf partition, with no separate global route search and with a finite-width room-clearance guarantee—has previously been described in equivalent form.

Table 1: Current regression coverage.

Test family	What is checked
Point arithmetic/indexing	Generic addition, subtraction, summation, immutable indexing, and mutable indexing in higher dimension.
BSP split preservation	Child boxes exactly partition the parent and only the selected coordinate changes; exercised in 1D, 2D, 4D, and 6D.
Split-axis selection	The generalized heuristic selects the largest eligible remaining dimension in a controlled 4D case.
Orthogonal routing	Start/end correctness, segment continuity, and at most one changed coordinate per segment; exercised in increasing and decreasing directions in 1D, 2D, 4D, and 6D.
Corridor bounds	Coordinate-wise min/max normalization and width expansion in 5D.
End-to-end generation	BSP generation, room construction, connectivity, routing, corridor construction, and export pipeline run without panic in 1D, 2D, 4D, and 5D.
Determinism/export	Equal seeded runs produce byte-identical OBJ output; generated OBJ contains vertices and faces.

9.2 Rectilinear path planning among axis-aligned obstacles

Orthogonal or rectilinear obstacle avoidance is also a mature subject. De Berg, van Kreveld, Nilsson, and Overmars give data structures and algorithms for shortest rectilinear paths among axis-parallel rectangles in the plane and boxes in higher-dimensional space [3]. Their problem is more general in obstacle placement and explicitly optimizes a path metric; the present construction does not claim shortest paths. Instead, it exploits an additional structural assumption absent from a general obstacle set: every room is inset inside a leaf of the same recursive axis-aligned partition through which the route is constructed. General graph-search methods such as Dijkstra’s algorithm and A* likewise solve broader path-selection problems [4, 5]. The algorithm studied here is intentionally more specialized: once the endpoints and BSP are given, it deterministically advances through partition boundaries rather than searching a separately constructed route graph.

9.3 BSP-based dungeon generation and corridor construction

BSP-based dungeon construction predates this manuscript as a standard constructive procedural-generation technique. Shaker et al. describe recursively partitioning a dungeon area, placing rooms in BSP leaves, and then using the BSP hierarchy to determine room connectivity; their

chapter notes that connecting rooms belonging to the same parent can reduce corridor overlap or intersection [6]. Baron compares several 2D dungeon-generation combinations, including BSP room placement paired with corridor-generation methods, and adapts procedural dungeon techniques to a 3D game setting [7]. More recently, Yap et al. present a Unity BSP dungeon generator in which a DFS-inspired corridor-generation method is used to improve connectivity reliability [8].

These sources make two points clear. First, using a BSP both to place rooms and to inform their connectivity is established practice. Second, practical BSP dungeon systems may still employ an additional corridor procedure, traversal, or search strategy. The present paper therefore does not claim novelty for BSP dungeon connectivity. Its candidate contribution is the specific geometric routing rule: traverse adjacent leaves in target-coordinate order while retaining a boundary coordinate through non-endpoint leaves, then derive a finite-width non-intersection condition directly from room-to-boundary clearance.

9.4 Guillotine partitions and dimensional generality

The axis-aligned BSPs used here are closely related to guillotine partitions: recursive partitions of an axis-aligned box by axis-aligned hyperplane cuts. Guillotine partitions have been studied independently of dungeon generation, including explicitly in d dimensions [9]. This is another reason the arbitrary-dimensional BSP implementation should not be read as a novelty claim. In this manuscript, the d -dimensional implementation is only a demonstration that the routing construction can be expressed in coordinate-indexed form.

9.5 Position of the present construction

The prior work above rules out several broad novelty claims. This paper does *not* claim to introduce BSPs, BSP-based motion planning, rectilinear obstacle avoidance, guillotine partitions, BSP dungeon generation, or hierarchical room connectivity. The narrower claim under investigation is:

Given rooms inset inside leaves of an axis-aligned BSP, the generating partition itself can be used as a deterministic orthogonal routing scaffold, without a separate global path search, while supporting a direct sufficient bound on finite corridor width from leaf-boundary clearance.

The literature search for this version located close work in each surrounding area but did not locate a source stating this same construction together with the boundary-preservation invariant and clearance theorem. That is evidence for, but not proof of, novelty: absence from a finite literature search is not evidence that no equivalent algorithm exists. Before a journal submission, the search should be repeated through citation chains from Tokuta, de Berg et al., the procedural-generation literature, and work on routing through rectangular or guillotine subdivisions.

10 Limitations and Open Proof Obligations

The current formulation assumes axis-aligned BSP cuts and axis-aligned rooms strictly inside leaves. It establishes a sufficient clearance condition, not a necessary or optimal one. It does

not claim globally shortest routes; shortest-path methods solve a different optimization problem [4, 5, 3]. Corridor–corridor intersections are a separate problem unless additional invariants are imposed. Degenerate boundary cases require an explicit convention about closed versus half-open leaf boxes in a discrete implementation.

Most importantly, the finite-width theorem depends on the intermediate-leaf boundary invariant. The implementation was designed around that invariant, and the current regression suite exercises behavior consistent with it, but v0.5 deliberately marks the full state-transition case proof as unfinished. Completing that proof against the exact routing implementation is the highest-priority formal revision.

11 Conclusion

This preprint isolates a constructive orthogonal routing algorithm for rooms embedded in a binary space partition. The central idea is to use the already-generated partition as the routing scaffold: move monotonically toward the target, cross adjacent leaves, and preserve a BSP boundary coordinate through non-endpoint regions. Termination follows from monotone movement through a finite partition. Subject to the boundary invariant, finite-width collision avoidance follows from the room-to-boundary clearance: an intersection would force the route–room distance to be no greater than the corridor half-width, contradicting a strictly larger minimum clearance.

The arbitrary-dimensional Rust implementation is intentionally secondary. Its purpose is to demonstrate that the algorithm can be represented using coordinate indices and interval operations without being tied to three named axes. The next substantive steps are to complete the boundary-invariant proof, add direct property tests for the mathematical invariants, and expand the prior-art comparison before journal submission.

Statement on AI-Assisted Tools

ChatGPT (OpenAI) was used during development of this manuscript as a research and writing aid, including discussion of proof structure, mathematical notation, manuscript organization, literature-search strategy, and editing. The routing algorithm and software implementation described here were developed by the author. AI-generated suggestions were not treated as authoritative sources; factual and scholarly claims are attributed to the references below. The author is solely responsible for the mathematical claims, software results, source verification, and final manuscript.

Version Note

This is preprint v0.5, September 2026. It is intended for early public circulation and feedback. Results and wording may change in later versions.

References

- [1] H. Fuchs, Z. M. Kedem, and B. F. Naylor, “On visible surface generation by a priori tree structures,” *ACM SIGGRAPH Computer Graphics*, vol. 14, no. 3, pp. 124–133, 1980. doi:10.1145/965105.807481.
- [2] A. O. Tokuta, “Motion planning using binary space partitioning,” in *Proceedings of the IEEE/RSJ International Workshop on Intelligent Robots and Systems (IROS)*, pp. 86–90, 1991. doi:10.1109/IROS.1991.174431.
- [3] M. de Berg, M. van Kreveld, B. J. Nilsson, and M. H. Overmars, “Shortest path queries in rectilinear worlds,” *International Journal of Computational Geometry & Applications*, vol. 2, no. 3, pp. 287–309, 1992. doi:10.1142/S0218195992000172.
- [4] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [5] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [6] N. Shaker, A. Liapis, J. Togelius, R. Lopes, and R. Bidarra, “Constructive generation methods for dungeons and levels,” in *Procedural Content Generation in Games*, N. Shaker, J. Togelius, and M. J. Nelson, Eds., Springer, 2016, pp. 31–55. doi:10.1007/978-3-319-42716-4_3.
- [7] J. R. Baron, “Procedural Dungeon Generation Analysis and Adaptation,” in *Proceedings of the 55th Annual Southeast Regional Conference (ACM SE ’17)*, pp. 168–171, 2017. doi:10.1145/3077286.3077566.
- [8] J. M. Yap, N. C. Woo, B. S. Tan, and K. S. Chong, “From Algorithm to Playable Space: A Technical Note on BSP-Based Dungeon Design,” *International Journal of Creative Multimedia*, vol. 6, no. 2, pp. 287–308, 2025. doi:10.33093/ijcm.2025.6.2.16.
- [9] A. Asinowski, G. Barequet, T. Mansour, and R. Y. Pinter, “Cut equivalence of d -dimensional guillotine partitions,” *Discrete Mathematics*, vol. 331, pp. 165–174, 2014. doi:10.1016/j.disc.2014.05.014.
- [10] The Rust Project Developers, “Generic parameters,” *The Rust Reference*. <https://doc.rust-lang.org/reference/items/generics.html>.